

b-secure — Complete System Requirements Document

Version: 1.0.0 **Date:** May 2026 **Type:** On-Demand Security Guard Platform

Inspiration: Rapido-style on-demand service model, applied to personal & property security

Table of Contents

1. [Product Overview](#)
 2. [System Architecture](#)
 3. [User App Requirements](#)
 4. [Guard App Requirements](#)
 5. [Admin Panel Requirements](#)
 6. [Backend Requirements \(Django\)](#)
 7. [Database Design](#)
 8. [Real-time Infrastructure](#)
 9. [Third-party Integrations](#)
 10. [Non-Functional Requirements](#)
 11. [Tech Stack](#)
 12. [Security & Compliance](#)
 13. [Development Phases & Timeline](#)
 14. [Deployment Architecture](#)
-

1. Product Overview

1.1 What is b-secure?

b-secure is a production-grade, on-demand security guard booking platform that connects individuals, families, businesses, and event organizers with verified, trained security professionals. Think of it as **Rapido or Uber — but for security services.**

A user opens the app, selects the type of protection they need (hourly escort, daily home guard, weekly office security, etc.), gets matched with a nearby verified guard, tracks them live, and pays seamlessly — all from their phone.

1.2 Core Value Proposition

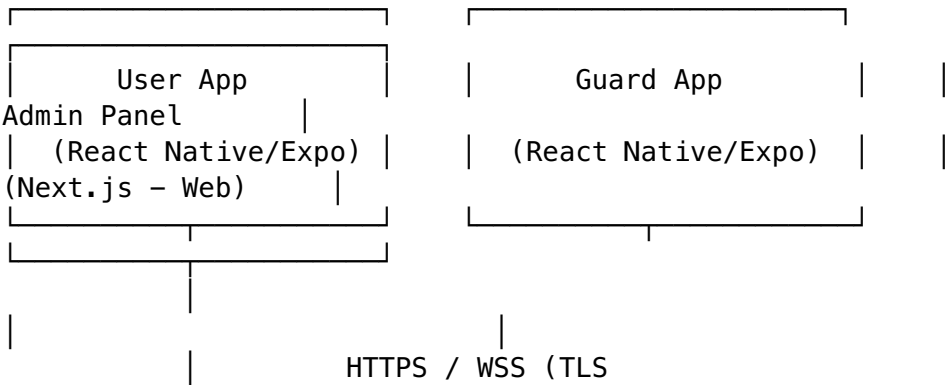
- **For Users:** Instant access to verified security guards with real-time tracking, SOS safety features, and flexible booking durations.
- **For Guards:** A reliable income platform with transparent earnings, scheduling flexibility, and a professional identity.
- **For Admins / Business Owners:** A centralised command center to monitor all platform activity, manage personnel, handle incidents, and drive revenue.

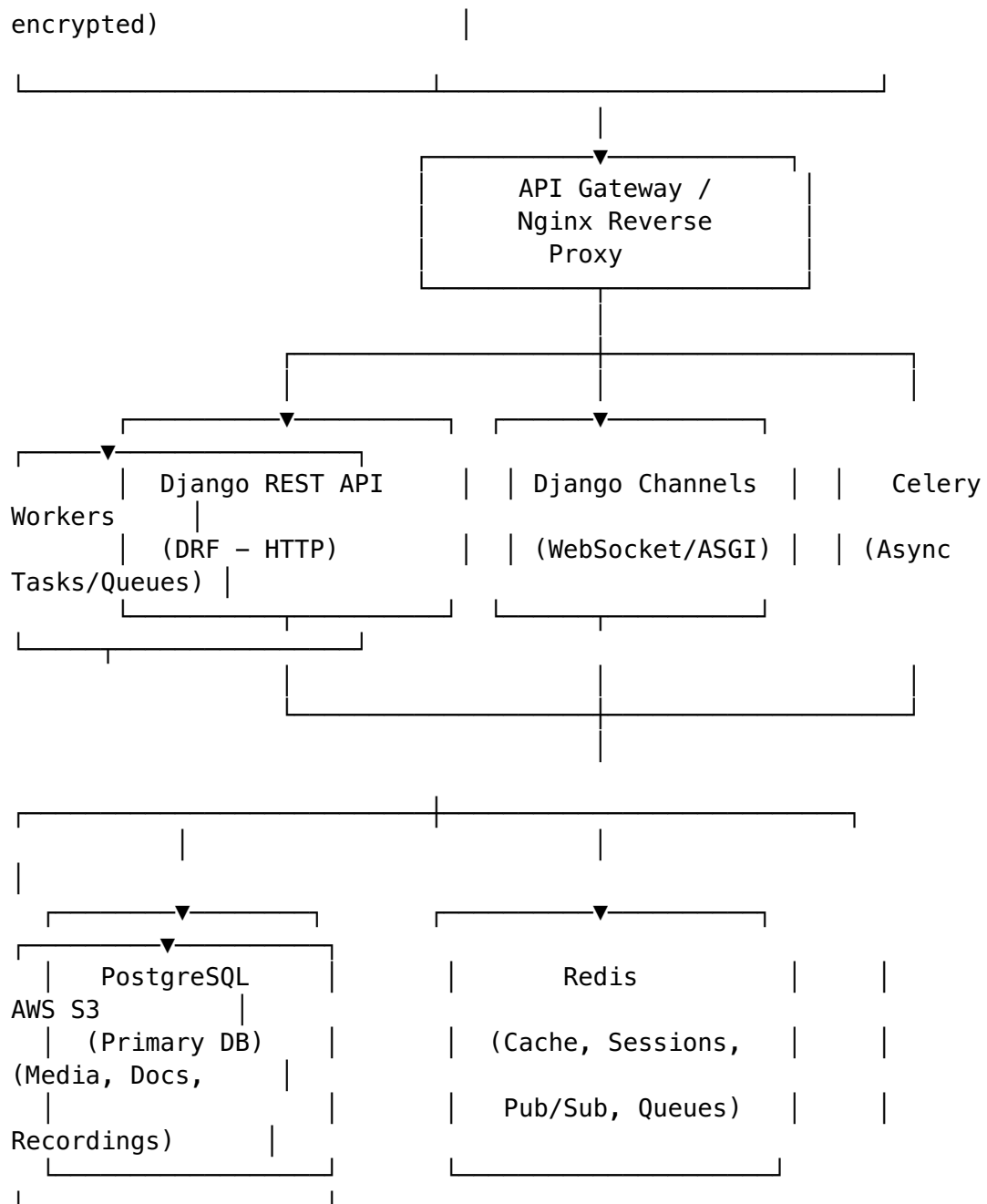
1.3 Platform Components

Component	Platform	Audience
User App	React Native (Expo)	End users seeking security
Guard App	React Native (Expo)	Security professionals
Admin Panel	React + Next.js (Web)	Platform operators & managers
Backend API	Django + Django REST Framework	All three clients
Real-time Server	Django Channels (WebSocket)	Live tracking & alerts

2. System Architecture

2.1 High-Level Architecture Diagram





2.2 Architecture Principles

- **Modular Django Apps:** Each domain (users, guards, bookings, payments, tracking, SOS) is its own Django app, loosely coupled with clean interfaces.
- **ASGI Server:** Django deployed via Daphne or Uvicorn to support both HTTP (REST) and WebSocket (Channels) connections simultaneously.
- **Async Task Queue:** Celery + Redis handles background tasks like notifications, payout processing, document verification webhooks, and report generation.
- **Stateless API:** JWT-based authentication ensures the API is stateless and horizontally scalable.

- **Event-Driven:** Critical events (SOS trigger, booking confirmed, guard offline) publish messages to Redis channels for real-time reactions.
-

3. User App Requirements

3.1 Onboarding & Authentication

Registration Flow

- Phone number entry → OTP verification via SMS (Twilio / MSG91)
- After OTP: collect full name, profile photo, email (optional)
- Optional: Sign in with Google or Apple ID
- On first login, prompt to set a **home address** and add **emergency contacts** (minimum 1 required for SOS feature)

Login

- Phone OTP (primary method)
- Biometric login (Face ID / Fingerprint) after first OTP login — using device Secure Enclave
- Session management: JWT access token (15 min expiry) + refresh token (30 days) stored in secure storage

Profile Management

- Edit name, photo, phone, email
 - Manage saved addresses (Home, Office, Custom)
 - Manage emergency contacts (name + phone, up to 5)
 - View account activity log
-

3.2 Service Discovery & Booking

Service Types

The platform offers security on four temporal scales:

Plan	Duration	Use Case	Billing
Hourly	Minimum 2 hours, up to 12	Late night escort, event, short-term	Per hour rate
Daily		Home guard, office,	Fixed daily rate

Plan	Duration	Use Case	Billing
Weekly	8-hour or 12-hour shift	construction site	Weekly package rate
	5 or 7 days	Extended travel, family protection	
Monthly	30 days	Permanent residence/office guard	Monthly subscription rate

Guard Type Selection

- **Unarmed Guard** — Trained security personnel, suitable for residential/retail
- **Armed Guard** — Licensed firearm carrier, suitable for high-risk scenarios (requires additional identity verification from user)
- **Female Guard** — For female users, schools, hospitals
- **Event Security** — Crowd management trained, multiple guards, event-specific package
- **Close Protection Officer (CPO)** — Executive-level protection, highly trained

Location & Scheduling

- **Immediate Booking:** Guard dispatched from current GPS location (similar to Rapido)
- **Advance Booking:** User picks a future date/time; system pre-assigns a guard
- **Recurring Booking:** Daily/Weekly/Monthly plans auto-renew unless cancelled
- Location input: current GPS, saved address, or manual map pin
- Service radius shown on map with available guards (anonymized dots until booking confirmed)

Guard Matching Algorithm

- Proximity: nearest available guard first
- Rating threshold: guards below 3.5 stars not shown for new bookings
- Specialization match: requested guard type vs guard's registered skill set
- Availability: only online guards with no active sessions shown
- Admin can override and manually assign from admin panel

Booking Confirmation Flow

1. User selects service → sees estimated price → confirms booking

2. System broadcasts request to nearby guards (within 5 km radius)
 3. First guard to accept gets the booking (30-second acceptance window)
 4. If no guard accepts: radius expands to 10 km, re-broadcast
 5. If still no acceptance: user notified, option to try again or schedule later
 6. On acceptance: user sees guard name, photo, rating, ETA, and live location on map
-

3.3 Live Tracking

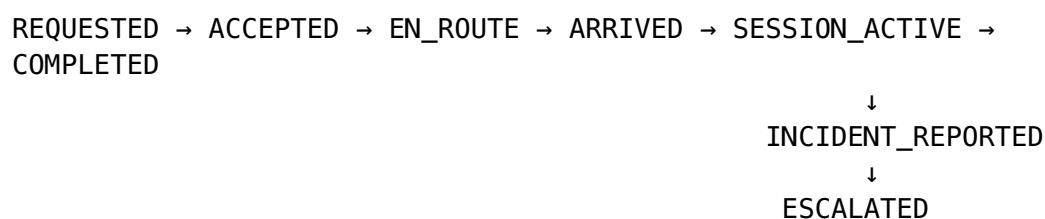
Map View (Active Session)

- Full-screen interactive map (Google Maps SDK / Mapbox)
- Guard's real-time location pinpoint, updating every 3–5 seconds
- User's location pinpoint
- Guard-to-user route line (polyline overlay)
- Live ETA countdown
- Session timer (elapsed time + remaining time for fixed plans)
- Guard status label: “En Route”, “Arrived”, “On Duty”, “Returning”

Location Accuracy

- GPS-primary with Wi-Fi and cell tower fallback
- Background location tracking (guard app keeps sharing even when screen is off)
- Geofencing alerts: notify user if guard moves beyond agreed perimeter

Session Lifecycle States



3.4 Safety Features

SOS / Panic Button

- Prominent red button always visible during active session (and optionally from home screen)
- Single tap opens confirmation (prevents accidental trigger), confirm within 3 seconds or auto-triggers

- On trigger:
 1. Instant push notification + SMS to all registered emergency contacts with user's live GPS coordinates
 2. Alert sent to b-secure control room (admin SOS dashboard goes live)
 3. Option to call emergency services (112 in India) directly from alert screen
 4. SOS session recorded: timestamp, location, audio (if permission granted)
 5. Guard is notified of user SOS immediately

Shake-to-SOS

- Accelerometer detects aggressive shake pattern (3 shakes within 2 seconds)
- Works even when app is in background (foreground service on Android)
- Same trigger flow as button SOS

Check-in System

- During long sessions (daily/weekly/monthly), guard must check in every configured interval (e.g. every 2 hours)
- If guard misses check-in, user gets alert + admin gets flag
- If 2 consecutive check-ins missed, auto-SOS escalation to admin

Session OTP Verification

- When guard arrives at user location, user generates a 4-digit OTP in their app
- Guard enters OTP in guard app to officially start the session
- This prevents fraudulent session starts and confirms physical presence
- Same OTP mechanism to end session (prevents premature end)

Incident Reporting

- User can file an incident report during or after a session
- Report types: misconduct, non-arrival, early departure, threatening behaviour, theft
- Photo/video evidence upload
- Incident logged in admin panel for immediate review

Audio/Video Evidence (Optional)

- User can enable session recording (audio only or video)
- Stored encrypted on AWS S3 with 30-day retention
- Accessible only by user and admin (with incident context)

- Clear consent prompt before enabling; guard is notified that recording is active
-

3.5 Payments

Wallet System

- In-app wallet with balance display
- Top-up methods: UPI (GPay, PhonePe, Paytm), Debit/Credit Card, Net Banking
- Auto-deduct from wallet on session completion
- Low balance alerts (configurable threshold)

Pricing Engine

- Base rate per guard type and service duration
- Surge pricing during high-demand periods (configurable by admin)
- Discounts: promo codes, first booking, referral credits
- Subscription plans: monthly plans with bundled hours at discounted rate

Billing & Invoices

- Auto-generated invoice after each completed session (PDF)
- Sent via email + downloadable from app
- Itemized: base fare + surge + taxes (GST 18%)
- Monthly statement for subscription users

Refunds & Cancellations

- Free cancellation: up to 30 minutes before scheduled start
- Partial refund: cancellation within 30–10 minutes of start
- No refund: cancellation after guard has arrived
- Disputed sessions: held in escrow pending admin resolution
- Refund credited back to wallet within 24 hours (or to source within 5–7 business days)

Payment Gateway Integration

- **Razorpay** (primary, India-focused): UPI, cards, wallets, EMI
 - **Stripe** (international users): cards, Apple Pay, Google Pay
 - PCI-DSS compliance handled at gateway level
-

3.6 Ratings & Reviews

- Post-session rating prompt (dismissible, reminded once)
 - 5-star scale with optional written review
 - Specific attribute ratings: Punctuality, Professionalism, Communication, Alertness
 - Flag review (for abuse/spam) — reviewed by admin
 - Guards can see their aggregate ratings but cannot see individual reviewer identity
 - Reviews feed into guard matching score
-

3.7 Notifications (User)

Event	Channel
Guard assigned	Push + SMS
Guard en route	Push
Guard arrived	Push
Session started	Push
Check-in reminder (missed by guard)	Push + SMS
Session ended	Push
Payment deducted	Push + Email
Invoice ready	Push + Email
SOS acknowledged	Push + SMS
Booking reminder (scheduled)	Push + SMS (1 hr before)
Promo / offers	Push (opt-in only)

4. Guard App Requirements

4.1 Onboarding & Verification

Registration

- Phone OTP login
- Personal details: name, DOB, gender, address
- Emergency contact (mandatory)

Document Upload & Verification

All documents uploaded as photos/PDFs, stored securely on S3, verified by admin:

Document	Purpose	Required
Government Photo ID (Aadhaar / Passport / Voter ID)	Identity verification	Mandatory
Police Verification Certificate	Background clearance	Mandatory
Security Guard License (PSARA)	Legal compliance	Mandatory
Training Certificate	Skill verification	Mandatory
Armed License	Only for armed guard role	Conditional
Live Selfie (at registration)	Face match with ID	Mandatory
Bank Account / UPI Details	Payout setup	Mandatory

Verification Workflow

1. Guard uploads documents → stored in S3
2. Admin reviews in admin panel (document viewer)
3. Admin approves / rejects with notes
4. Guard notified of status via push + SMS
5. Re-submission allowed for rejected documents (with reason shown)
6. Only fully verified guards go “Active” and appear for booking

Periodic Re-verification

- Annual police verification renewal
- License expiry tracking — admin alerted 30 days before expiry
- Guard auto-suspended if critical documents expire

4.2 Availability & Schedule Management

- **Online / Offline Toggle:** Guard can go online (available for bookings) or offline (not accepting)
- **Working Hours:** Set preferred working hours (e.g. 8 AM–8 PM); system respects this for advance bookings
- **Blackout Dates:** Mark unavailable dates

- **Maximum Active Sessions:** Guards handle one session at a time (configurable by admin for event guards)
 - **Booking Request Screen:** 30-second timer with client details (first name, service type, duration, location area — not exact address until accepted)
-

4.3 Active Session Management

Acceptance Flow

1. Booking request appears → guard sees: service type, duration, distance, estimated earnings
2. Accept or Decline within 30 seconds
3. On accept: full client location revealed + navigation starts (Google Maps deep link)
4. Guard begins travelling to client

Session Start

- Guard arrives at client location
- Guard taps “I’ve Arrived” in app
- Client generates OTP → guard enters OTP → session officially starts
- Location broadcasting begins (every 3–5 seconds to server)

During Session

- Session timer running
- Client contact button (masked call)
- In-app chat with client
- Check-in button (guard taps to confirm they are active)
- Incident report button (guard can report if client is threatening or in danger)
- Distress button (guard SOS — see below)

Session End

- Client generates end OTP → guard enters → session ends
 - Or: session auto-ends at planned duration end time (with 10-minute warning)
 - Summary screen: duration, earnings for this session
-

4.4 Guard Safety Features

Guard Distress Alert

- Guard presses distress button if they feel threatened
- Platform control room alerted immediately with guard's GPS location
- Option to call police from the alert screen
- Client notified if guard presses distress

Dead Man's Switch

- If guard's phone goes completely offline during an active session (battery dead, app crash, network loss) for more than 10 minutes:
 - Admin alerted
 - Client alerted with last known location of guard
 - Auto-escalation protocol triggered
-

4.5 Earnings & Payouts

Earnings Dashboard

- Today's earnings, this week, this month
- Earnings per session (with session details)
- Total hours worked
- Ratings summary

Payout System

- Earnings calculated: (hourly rate × hours) – platform commission (e.g. 15–20%)
- Payout schedule: Daily settlement (for completed sessions), or Weekly on Fridays
- Payout methods: Bank transfer (NEFT/IMPS) or UPI
- Minimum payout threshold: ₹200
- Transaction history with timestamps and reference IDs

Tax Documents

- Monthly earnings statement (downloadable PDF)
 - Annual income summary for ITR filing
 - TDS deduction details if applicable (threshold-based)
-

4.6 Guard Notifications

Event	Channel
New booking request	Push (high priority) + vibration
Booking cancelled before start	Push
Session about to expire (15 min warning)	Push
Payout processed	Push + SMS
Document approved / rejected	Push + SMS
License expiry reminder	Push + SMS
Check-in reminder	Push
SOS alert from client	Push (critical)

5. Admin Panel (Centralised System)

The Admin Panel is a web application (Next.js) that gives the b-secure operations team full visibility and control over the entire platform.

5.1 Dashboard (Home Screen)

Live Operations Map

- Full-screen map (Google Maps) showing:
 - All online guards (green pins)
 - All active sessions (animated route lines connecting user and guard)
 - All SOS alerts (red pulsing pins — highest priority)
 - Recent incidents (yellow pins)
- Click any pin to see details panel on the right

KPI Cards (Real-time, auto-refreshing every 30 seconds)

- Active sessions right now
- Guards online right now
- Users currently in app
- Total bookings today
- Revenue today / this week / this month
- Open SOS alerts
- Pending guard approvals

Activity Feed

- Live feed of recent platform events: new bookings, completions, cancellations, SOS triggers, payments
-

5.2 User Management

- **List View:** Paginated table with search, filter by date joined, status (active / suspended / banned)
 - **User Detail Page:**
 - Profile info, contact details
 - Booking history (with ability to view each session's details and map replay)
 - Payment history and wallet balance
 - SOS history
 - Support tickets
 - Risk flags (if any automated flags triggered)
 - **Actions:** Suspend (temporary), Ban (permanent), Reset password, Refund booking, Add wallet credit
-

5.3 Guard Management

- **List View:** Filter by status (pending / active / suspended / inactive), guard type, rating, location
- **Guard Detail Page:**
 - Profile, documents viewer (PDF/image viewer in-panel)
 - Verification status for each document
 - Session history with map replays
 - Earnings summary
 - Ratings & reviews received
 - Incident history
 - License expiry dates
- **Actions:** Approve/Reject documents, Activate/Suspend/Ban guard, Override payout, Add internal notes

Document Verification Queue

- Dedicated page showing all guards with pending document reviews
 - Admin can approve/reject each document individually with remarks
 - Bulk approval for renewals
-

5.4 Booking Management

- **List View:** Filter by status (requested / active / completed / cancelled / disputed), service type, date range
 - **Booking Detail Page:**
 - Full timeline of booking lifecycle (with timestamps)
 - Map replay of the session (playback of guard's route)
 - User and guard involved
 - Payment breakdown
 - Any incidents or SOS events during this booking
 - **Actions:** Cancel booking, Reassign guard, Issue refund, Mark as disputed, Add admin notes
-

5.5 SOS & Incident Control Room

This is a mission-critical, always-visible section for the operations team.

SOS Dashboard

- All active SOS alerts shown prominently at the top of screen
- Each alert card: user name, guard name, GPS location, time since triggered, escalation status
- Click to open full SOS detail: live map of user's location, contact options, escalation log
- **One-click call to user** (via Twilio from admin panel)
- **One-click call to emergency services** (with user's coordinates pre-filled)
- Assign a control room agent to handle each SOS
- Resolution logging: what action was taken, outcome

Incident Log

- All filed incident reports
 - Filter by type, severity, date, resolution status
 - Assign to internal team for investigation
 - Link incident to guard's profile (affects rating and standing)
 - Close incident with resolution notes
-

5.6 Financial Management

Revenue Overview

- Gross revenue, platform commission, guard payouts
- Breakdown by service type, region, time period

- Month-over-month / year-over-year charts

Payout Management

- List of pending guard payouts
- Batch payout approval (admin reviews and approves)
- Manual payout trigger
- Payout history with bank transfer reference numbers

Refund Management

- Disputed session refunds queue
- Approve / partial approve / deny with reason
- Automated refund triggers for certain conditions

Commission Configuration

- Set platform fee percentage per guard type
 - Set surge pricing multipliers by time/location
 - Set promo code rules (admin creates and manages promo codes)
-

5.7 Notifications & Communication

- **Broadcast Push:** Send push notification to all users, all guards, or a filtered segment
 - **Individual Message:** Send in-app message to specific user or guard
 - **SMS Broadcast:** For critical announcements (e.g. service disruption)
 - **Email Campaigns:** Integration with SendGrid for transactional and promotional emails
 - **Template Management:** Edit SMS / push / email templates used by the system
-

5.8 Analytics & Reporting

Booking Analytics

- Bookings by service type, duration, location heatmap
- Peak demand times (hourly heatmap)
- Cancellation rate and reasons
- Funnel analysis: request → accepted → completed

Guard Performance Analytics

- Average ratings over time
- On-time arrival rate
- Acceptance rate (bookings accepted vs declined)
- Incident rate per guard

Financial Reports

- Revenue reports (daily / weekly / monthly / custom range)
- Tax reports (GST filing support)
- Guard payout reports
- Exportable as CSV and PDF

User Analytics

- Retention: users who booked again within 30 days
 - Lifetime value (LTV) per user
 - Churn analysis
 - Geographic distribution of users
-

6. Backend Requirements (Django)

6.1 Project Structure

```
bsecure_backend/
├── manage.py
├── config/
│   ├── settings/
│   │   ├── base.py          # Shared settings
│   │   └── development.py   # Dev-specific (DEBUG=True, SQLite
│   │   option)
│   ├── production.py        # Prod (PostgreSQL, S3, Sentry, etc.)
│   ├── urls.py              # Root URL router
│   ├── asgi.py              # ASGI entry point (HTTP + WebSocket)
│   └── wsgi.py
├── apps/
│   ├── authentication/      # OTP, JWT, social auth
│   └── users/               # User profiles, addresses, emergency
├── contacts
│   └── guards/              # Guard profiles, documents,
├── availability
│   ├── bookings/           # Booking lifecycle management
│   └── tracking/            # Real-time location (Channels
├── WebSocket consumers)
│   ├── payments/           # Wallet, transactions, payouts
│   └── notifications/      # Push, SMS, email dispatch
```

		sos/	# SOS alerts, incidents, escalation
		reviews/	# Ratings and reviews
		admin_panel/	# Admin-specific APIs and serializers
		analytics/	# Aggregated analytics queries
		utils/	
			permissions.py # Custom DRF permission classes
			pagination.py # Custom pagination
			validators.py # Phone, document, geo validators
			helpers.py # Common utility functions
		celery_app.py	# Celery configuration
		requirements/	
			base.txt
			development.txt
			production.txt

6.2 Django Apps Breakdown

authentication App

- **Models:** OTPToken (phone, otp_code, expires_at, is_used), RefreshToken
- **Views/APIs:**
 - POST /api/auth/send-otp/ — generates OTP, sends via SMS
 - POST /api/auth/verify-otp/ — validates OTP, returns JWT pair
 - POST /api/auth/refresh/ — refresh access token
 - POST /api/auth/logout/ — blacklist refresh token
 - POST /api/auth/social/google/ — Google OAuth2 token exchange
 - POST /api/auth/social/apple/ — Apple Sign In
- **Libraries:** django-rest-framework-simplejwt, social-django (for OAuth)
- **Security:** OTP expires in 5 minutes, max 3 attempts before lock, rate-limited per phone number

users App

- **Models:** UserProfile (extends AbstractUser), Address, EmergencyContact
- **APIs:**
 - GET/PUT /api/users/me/ — own profile
 - GET/POST/DELETE /api/users/me/addresses/
 - GET/POST/DELETE /api/users/me/emergency-contacts/
 - GET /api/users/me/bookings/ — booking history
 - GET /api/users/me/wallet/ — wallet balance and transactions

guards App

- **Models:** GuardProfile, GuardDocument, GuardAvailability, GuardSkill
- **APIs:**
 - GET/PUT /api/guards/me/ — guard's own profile
 - POST /api/guards/me/documents/ — upload document

- GET /api/guards/me/documents/ — list own documents with verification status
- PUT /api/guards/me/availability/ — update online/offline status, working hours
- GET /api/guards/me/earnings/ — earnings summary
- GET /api/guards/nearby/ — (used by booking service) get nearby available guards
- **Document Verification States:** PENDING → UNDER_REVIEW → APPROVED / REJECTED

bookings App

- **Models:** Booking, BookingRequest (broadcast state), Session
- **Booking States (FSM):** REQUESTED → BROADCAST → ACCEPTED → EN_ROUTE → ARRIVED → ACTIVE → COMPLETED / CANCELLED / DISPUTED
- **APIs:**
 - POST /api/bookings/ — create new booking
 - GET /api/bookings/{id}/ — booking detail
 - POST /api/bookings/{id}/cancel/ — cancel booking
 - POST /api/bookings/{id}/generate-otp/ — user generates start/end OTP
 - POST /api/bookings/{id}/verify-otp/ — guard verifies OTP to start/end
 - GET /api/guards/me/booking-requests/ — guard sees pending requests
 - POST /api/guards/me/booking-requests/{id}/accept/
 - POST /api/guards/me/booking-requests/{id}/decline/
- **Celery Tasks:**
 - broadcast_booking_request — broadcast to nearby guards in rings (5km → 10km)
 - expire_booking_request — auto-cancel if no guard found within timeout
 - check_session_checkin — periodic check if guard has checked in

tracking App

- **Django Channels WebSocket Consumers:**
 - TrackingConsumer — handles real-time location updates
 - Connection groups: session_{booking_id} — user, guard, admin all join this group
 - Guard sends location update → server validates → broadcasts to group

- **Models:** LocationSnapshot (guard_id, lat, lng, accuracy, timestamp, booking_id)
 - Time-series optimized table (consider TimescaleDB extension for PostgreSQL)
- **APIs:**
 - GET /api/tracking/sessions/{booking_id}/history/ — replay location history of a session

- **WebSocket Protocol:**

```
// Guard sends (every 3-5 seconds):
{ "type": "location_update", "lat": 12.9716, "lng": 77.5946,
  "accuracy": 5.2 }

// Server broadcasts to session group:
{ "type": "guard_location", "lat": 12.9716, "lng": 77.5946,
  "accuracy": 5.2, "timestamp": "2026-05-28T10:30:00Z",
  "eta_seconds": 240 }

// Status updates:
{ "type": "session_status_change", "status": "ARRIVED",
  "timestamp": "..." }
```

payments App

- **Models:** Wallet, Transaction, PaymentOrder, Payout
- **Transaction Types:** TOPUP, BOOKING_DEBIT, REFUND, PAYOUT, PROMO_CREDIT
- **APIs:**
 - GET /api/payments/wallet/ — balance and recent transactions
 - POST /api/payments/wallet/topup/ — initiate Razorpay/Stripe order
 - POST /api/payments/webhook/razorpay/ — Razorpay payment confirmation webhook
 - POST /api/payments/webhook/stripe/ — Stripe payment confirmation webhook
 - GET /api/payments/invoices/{booking_id}/ — download invoice PDF
 - GET /api/guards/me/payouts/ — guard payout history
- **Celery Tasks:**
 - process_session_payment — triggered on session completion
 - process_guard_payout — runs on payout schedule (daily/weekly)
 - generate_invoice_pdf — async PDF generation (ReportLab / WeasyPrint)

notifications App

- **Models:** NotificationLog (recipient, channel, content, sent_at, status)
- **Notification Channels:**
 - **Push:** Firebase Cloud Messaging (FCM) via pyfcm library
 - **SMS:** Twilio or MSG91 via REST API
 - **Email:** SendGrid via sendgrid-python
 - **In-App:** stored in DB, fetched on app open
- **APIs:**
 - GET /api/notifications/ — user's notification inbox (in-app)
 - POST /api/notifications/{id}/read/ — mark as read
 - PUT /api/users/me/notification-preferences/ — opt in/out of channels
- **Celery Tasks:** All notification sends are async Celery tasks to avoid blocking API responses

sos App

- **Models:** SOSAlert (user, booking, location_lat, location_lng, triggered_at, status, resolved_by, resolution_notes), Incident
- **SOS States:** TRIGGERED → ACKNOWLEDGED → RESPONDING → RESOLVED
- **APIs:**
 - POST /api/sos/trigger/ — user triggers SOS (auth required)
 - POST /api/sos/{id}/acknowledge/ — admin acknowledges
 - POST /api/sos/{id}/resolve/ — admin resolves with notes
 - POST /api/incidents/ — user or guard files incident report
 - GET /api/incidents/ — user's own incidents
- **On SOS Trigger (synchronous + async):**
 1. Immediately persist SOS record (synchronous — must be instant)
 2. Async: send push + SMS to emergency contacts (Celery task)
 3. Async: push to admin SOS dashboard via WebSocket
 4. Async: if booking active, notify guard

reviews App

- **Models:** Review (booking, reviewer, reviewee, rating, comment, attribute_ratings)
- **APIs:**
 - POST /api/reviews/ — submit review (only after session completed)
 - GET /api/guards/{id}/reviews/ — public reviews for a guard
 - PUT /api/reviews/{id}/flag/ — flag a review

admin_panel App

- Separate serializers and views for admin use (not exposed to regular users via routing)

- All admin APIs protected by `IsAdminUser` permission class
- **Key APIs:**
 - GET `/api/admin/dashboard/stats/` — KPI metrics
 - GET `/api/admin/live-map/` — all active session locations
 - GET `/api/admin/users/`
 - GET `/api/admin/guards/`
 - POST `/api/admin/guards/{id}/verify-document/`
 - GET `/api/admin/bookings/`
 - GET `/api/admin/sos-alerts/`
 - GET `/api/admin/analytics/bookings/`
 - GET `/api/admin/analytics/revenue/`
 - POST `/api/admin/notifications/broadcast/`

6.3 Django REST Framework Configuration

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
    'DEFAULT_PAGINATION_CLASS':
        'utils.pagination.StandardResultsSetPagination',
    'PAGE_SIZE': 20,
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle',
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '20/minute',
        'user': '200/minute',
        'otp': '5/minute',          # Custom throttle for OTP
                                   # endpoint
    },
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.SearchFilter',
        'rest_framework.filters.OrderingFilter',
    ],
    'EXCEPTION_HANDLER':
        'utils.exceptions.custom_exception_handler',
}
```

6.4 Key Django Settings (Production)

```
# Database
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
```

```

        'NAME': env('DB_NAME'),
        'USER': env('DB_USER'),
        'PASSWORD': env('DB_PASSWORD'),
        'HOST': env('DB_HOST'),
        'PORT': '5432',
        'CONN_MAX_AGE': 60,
        'OPTIONS': {'sslmode': 'require'},
    }
}

# Channel Layers (Redis for WebSocket)
CHANNEL_LAYERS = {
    'default': {
        'BACKEND': 'channels_redis.core.RedisChannelLayer',
        'CONFIG': {
            'hosts': [env('REDIS_URL')],
            'capacity': 1500,
            'expiry': 10,
        },
    },
}

# Celery
CELERY_BROKER_URL = env('REDIS_URL')
CELERY_RESULT_BACKEND = env('REDIS_URL')
CELERY_TASK_SERIALIZER = 'json'
CELERY_ACCEPT_CONTENT = ['json']
CELERY_TIMEZONE = 'Asia/Kolkata'

# File Storage
DEFAULT_FILE_STORAGE = 'storages.backends.s3boto3.S3Boto3Storage'
AWS_STORAGE_BUCKET_NAME = env('AWS_S3_BUCKET')
AWS_S3_REGION_NAME = 'ap-south-1'
AWS_S3_FILE_OVERWRITE = False
AWS_DEFAULT_ACL = 'private' # All files private by default

# Caching
CACHES = {
    'default': {
        'BACKEND': 'django_redis.cache.RedisCache',
        'LOCATION': env('REDIS_URL'),
        'OPTIONS': {'CLIENT_CLASS':
            'django_redis.client.DefaultClient'},
        'TIMEOUT': 300,
    }
}

# CORS (allow mobile apps and admin web)
CORS_ALLOWED_ORIGINS = env.list('CORS_ALLOWED_ORIGINS')
CORS_ALLOW_CREDENTIALS = True

```

7. Database Design

7.1 Core Tables / Models

users_userprofile

- └─ id (UUID, PK)
- └─ phone_number (unique, indexed)
- └─ full_name
- └─ email
- └─ profile_photo_url
- └─ date_of_birth
- └─ gender
- └─ is_active
- └─ is_suspended
- └─ fcm_token (for push notifications)
- └─ created_at
- └─ updated_at

users_address

- └─ id, user (FK), label (HOME/OFFICE/OTHER)
- └─ line1, line2, city, state, pincode
- └─ latitude, longitude
- └─ is_default

users_emergencycontact

- └─ id, user (FK), name, phone_number, relationship
- └─ is_primary

guards_guardprofile

- └─ id (UUID, PK)
- └─ user (OneToOne FK to UserProfile)
- └─ guard_type (UNARMED/ARMED/FEMALE/CPO/EVENT)
- └─ verification_status (PENDING/ACTIVE/SUSPENDED/BANNED)
- └─ years_of_experience
- └─ bio
- └─ is_online
- └─ current_latitude, current_longitude
- └─ last_location_update
- └─ average_rating (cached, updated on new review)
- └─ total_sessions_completed
- └─ fcm_token
- └─ bank_account_number, bank_ifsc, upi_id
- └─ created_at

guards_guardddocument

- └─ id, guard (FK)
- └─ document_type (GOVT_ID/POLICE_CERT/PSARA/TRAINING/ARMED_LICENSE)
- └─ file_url (S3 path)
- └─ status (PENDING/UNDER_REVIEW/APPROVED/REJECTED)
- └─ reviewer (FK to admin User, nullable)
- └─ review_notes

- expiry_date (nullable)
- uploaded_at, reviewed_at

bookings_booking

- id (UUID, PK)
- user (FK), guard (FK, nullable until accepted)
- service_type (HOURLY/DAILY/WEEKLY/MONTHLY)
- guard_type_requested
- status (REQUESTED/BROADCAST/ACCEPTED/EN_ROUTE/ARRIVED/ACTIVE/COMPLETED/CANCELLED/DISPUTED)
- scheduled_start, scheduled_end
- actual_start, actual_end
- location_address, location_latitude, location_longitude
- total_amount, platform_fee, guard_earnings
- start_otp, end_otp (hashed)
- cancellation_reason
- is_recurring, recurrence_rule (JSON)
- created_at

tracking_locationsnapshot

- id, booking (FK), guard (FK)
- latitude, longitude, accuracy
- timestamp (indexed – time-series)
- speed, bearing (optional, for route replay)

payments_wallet

- id, user (FK or Guard FK)
- balance (Decimal)
- updated_at

payments_transaction

- id (UUID), wallet (FK)
- transaction_type (TOPUP/DEBIT/REFUND/PROMO)
- amount, balance_after
- booking (FK, nullable)
- payment_gateway_order_id
- payment_gateway_payment_id
- status (PENDING/SUCCESS/FAILED/REFUNDED)
- created_at

sos_sosalert

- id (UUID, PK)
- user (FK), booking (FK, nullable)
- trigger_method (BUTTON/SHAKE/AUTO)
- latitude, longitude
- status (TRIGGERED/ACKNOWLEDGED/RESPONDING/RESOLVED)
- assigned_admin (FK, nullable)
- resolution_notes
- triggered_at, resolved_at

reviews_review

- id, booking (FK, unique), reviewer (FK), guard (FK)
- overall_rating (1-5)
- punctuality_rating, professionalism_rating,

```
communication_rating, alertness_rating
├── comment
├── is_flagged, flag_reason
└── created_at
```

7.2 Indexing Strategy

```
-- Geospatial index for guard proximity queries
CREATE INDEX idx_guard_location ON guards_guardprofile USING GIST
(
    ST_SetSRID(ST_MakePoint(current_longitude, current_latitude),
        4326)
);
-- Requires PostGIS extension

-- Time-series index for location snapshots
CREATE INDEX idx_location_booking_time ON
    tracking_locationsnapshot (booking_id, timestamp DESC);

-- Booking status queries
CREATE INDEX idx_booking_status ON bookings_booking (status,
    created_at DESC);
CREATE INDEX idx_booking_user ON bookings_booking (user_id,
    created_at DESC);
CREATE INDEX idx_booking_guard ON bookings_booking (guard_id,
    status);

-- SOS active alerts
CREATE INDEX idx_sos_status ON sos_sosalert (status) WHERE
    status != 'RESOLVED';
```

7.3 PostGIS for Geospatial Queries

```
# Finding guards within X km of a location
from django.contrib.gis.db.models.functions import Distance
from django.contrib.gis.geos import Point
from django.contrib.gis.measure import D

user_location = Point(lng, lat, srid=4326)

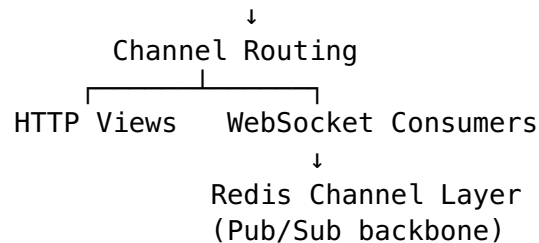
nearby_guards = GuardProfile.objects.filter(
    is_online=True,
    verification_status='ACTIVE',
    guard_type=requested_type,
    location__distance_lte=(user_location, D(km=5))
).annotate(
    distance=Distance('location', user_location)
).order_by('distance')[:10]
```

8. Real-time Infrastructure

8.1 Django Channels Setup

ASGI Stack:

Nginx → Daphne (ASGI Server) → Django Channels



8.2 WebSocket Consumers

TrackingConsumer

- URL: `wss://api.bsecure.in/ws/tracking/{booking_id}/`
- Authentication: JWT token passed as query param `?token=...`
- Groups: user joins `session_{booking_id}`, guard joins same group, admin joins `admin_live_map`
- Guard sends location every 3–5 seconds
- Server computes ETA using Google Maps Distance Matrix API
- Server broadcasts to group, admin map group, and persists to DB

SOSConsumer

- URL: `wss://api.bsecure.in/ws/sos/`
- Admin control room subscribes to this
- All SOS triggers broadcast here in real-time to connected admins

AdminDashboardConsumer

- URL: `wss://api.bsecure.in/ws/admin/dashboard/`
- Pushes live KPI updates, new bookings, new SOS alerts to admin panel

8.3 Celery Task Queue Structure

Queues:

- | | |
|---------------|--|
| high_priority | → SOS alerts, booking request broadcasts, OTP sends |
| default | → Session payments, notifications, check-in monitoring |
| low_priority | → Report generation, analytics aggregation, PDF invoices |

scheduled
expiry checks) → Periodic tasks (payout processing, license

9. Third-party Integrations

Category	Service	Purpose	Library
SMS OTP	Twilio / MSG91	OTP delivery, alerts to emergency contacts	twilio / REST API
Push Notifications	Firebase Cloud Messaging	All push notifications to mobile apps	pyfcm / firebase-admin
Maps & Geo	Google Maps Platform	Geocoding, Distance Matrix, Static Maps	googlemaps
Navigation	Google Maps / Mapbox	Turn-by-turn for guard app	Mobile SDK
Payments	Razorpay	India payments (UPI, cards, wallets)	razorpay
Payments	Stripe	International card payments	stripe
File Storage	AWS S3	Documents, photos, recordings	boto3, django-storages
Email	SendGrid	Transactional emails, invoices	sendgrid
Identity Verification	IDfy / AuthBridge	Background check API	REST API
Crash Reporting	Sentry	Error tracking (backend + mobile)	sentry-sdk
APM	Datadog / New Relic	Performance monitoring	Agent
PDF Generation	WeasyPrint / ReportLab	Invoice and report PDFs	weasyprint
Geospatial DB	PostGIS Elasticsearch	Proximity queries	Django GIS

Category	Service	Purpose	Library
Search (optional)		Guard search, full-text log search	elasticsearch- dsl

10. Non-Functional Requirements

10.1 Performance

Metric	Target
API response time (p95)	< 300ms
Location update propagation (WebSocket)	< 500ms end-to-end
App cold start	< 3 seconds
Search/nearby guard query	< 200ms
SOS trigger to admin notification	< 2 seconds
Concurrent active sessions supported	10,000+
Database query time (p99)	< 100ms

10.2 Scalability

- Django API: horizontal scaling behind load balancer (AWS ALB)
- Daphne/Channels: multiple instances, all using shared Redis channel layer
- Celery: auto-scaling worker pool based on queue depth
- PostgreSQL: read replicas for analytics queries, write to primary
- Redis: Redis Cluster for high availability and throughput
- S3: inherently scalable, CDN (CloudFront) for media delivery

10.3 Availability

- Target: **99.9% uptime** (< 9 hours downtime/year)
- Multi-AZ deployment on AWS
- Database: RDS with Multi-AZ failover (automatic < 60 seconds)
- Redis: ElastiCache with Multi-AZ
- Health checks on all services with auto-restart policies
- Graceful degradation: if real-time tracking fails, app continues to function for bookings/payments

10.4 Security

Area	Implementation
Authentication	JWT (short-lived access + refresh token rotation)
Transport	HTTPS everywhere, WSS for WebSocket, HSTS headers
API Security	Rate limiting, input validation, parameterized queries (ORM)
Secrets Management	AWS Secrets Manager / environment variables (never in code)
File Access	S3 pre-signed URLs (expire in 15 min) for document downloads
Password	Django's PBKDF2 + SHA256 (or Argon2 via django[argon2])
Admin Access	IP whitelist + 2FA for admin panel
Data Encryption	Encryption at rest (AWS RDS encrypted volumes, S3 SSE)
OWASP Top 10	Django's built-in protections (CSRF, XSS, SQL injection via ORM)
Audit Logging	All admin actions logged to immutable audit table

10.5 Data Privacy & Compliance

- **DPDP Act 2023 (India):** User consent for data collection, right to erasure, data minimization
- **GDPR (for EU users):** Cookie consent, data portability, right to be forgotten
- **PCI-DSS:** Handled at payment gateway level (never store raw card data)
- **PSARA Act (India):** Security agency regulatory compliance
- Location data retention: raw location snapshots retained for 90 days, then aggregated/anonymized
- Audio/video recordings: 30-day retention, user-accessible, encrypted

10.6 Testing Requirements

Test Type	Tool	Coverage Target
Unit Tests	pytest-django	80% backend code coverage

Test Type	Tool	Coverage Target
API Integration Tests	pytest + DRF test client	All endpoints
WebSocket Tests	pytest-asyncio + Channels testing utils	Tracking, SOS consumers
Mobile E2E Tests	Detox	Critical user flows
Load Testing	Locust	1000 concurrent users simulation
Security Testing	OWASP ZAP	Pre-release scan

11. Tech Stack

11.1 Complete Stack Summary

Layer	Technology	Version
User App	React Native (Expo)	SDK 51+
Guard App	React Native (Expo)	SDK 51+
Admin Panel	Next.js	14+
Backend API	Django + Django REST Framework	Django 5.x, DRF 3.15+
Real-time	Django Channels	4.x
ASGI Server	Daphne	4.x
Task Queue	Celery + Redis (BullMQ-style)	Celery 5.x
Primary Database	PostgreSQL + PostGIS	PG 16+
Cache / Pub-Sub	Redis	7.x
File Storage	AWS S3	—
CDN	AWS CloudFront	—
Containerization	Docker + Docker Compose	—
Orchestration	Kubernetes (AWS EKS)	—
CI/CD	GitHub Actions	—
Monitoring	Sentry + Datadog	—
Maps	Google Maps Platform	—

11.2 Key Python Libraries

```
# requirements/base.txt
Django==5.0.6
django-rest-framework==3.15.2
django-rest-framework-simplejwt==5.3.1
django-channels==4.1.0
channels-redis==4.2.0
daphne==4.1.2
celery==5.4.0
redis==5.0.7
psycopg2-binary==2.9.9
django-cors-headers==4.4.0
django-filter==24.2
django-storages[s3]==1.14.3
boto3==1.34.144
Pillow==10.4.0
phonenumbers==8.13.42
pyfcm==2.0.7
twilio==9.2.3
razorpay==1.4.1
stripe==10.6.0
sendgrid==6.11.0
sentry-sdk[django]==2.10.0
WeasyPrint==62.3
django-environ==0.11.2
django-rest-framework-gis==1.0
GDAL==3.8.4
social-auth-app-django==5.4.1
django-redis==5.4.0
```

12. Security & Compliance

12.1 PSARA Act Compliance (India)

- The Private Security Agencies Regulation Act (PSARA) requires:
 - All security agencies to be licensed in each state of operation
 - Guards must have police verification
 - Guards must be trained per PSARA guidelines
 - Platform must maintain records of all guards and their assignments
- b-secure's admin panel must be able to generate PSARA compliance reports on demand

12.2 Data Security Architecture

User Data Flow:

Mobile App → (TLS 1.3) → Nginx → Django → PostgreSQL (encrypted at rest)

- S3 (SSE-S3 encryption)
- Redis (ephemeral, no

PII stored)

Admin Access:

Admin Panel → (VPN + TLS) → Nginx → Django Admin API

All admin actions → Audit Log Table (append-only)

12.3 Incident Response Plan

- **Security breach:** 72-hour notification to affected users (DPDP Act requirement)
 - **SOS failure:** Dedicated runbook, manual escalation to local emergency services
 - **Payment fraud:** Auto-freeze transaction, notify user, investigate within 24 hours
 - **Data loss:** RDS automated backups (daily), point-in-time recovery (35-day window)
-

13. Development Phases & Timeline

Phase 1 — MVP (Months 1–4)

Goal: Core booking flow end-to-end, live tracking, basic payments

Deliverables: - [] Django backend scaffolding, auth, user & guard models - [] Guard document upload and admin verification flow - [] Booking creation, request broadcast, guard accept/decline - [] Session lifecycle with OTP start/end - [] Basic live tracking (WebSocket) - [] Razorpay wallet top-up and session payment - [] User App: auth, booking flow, live map, payment - [] Guard App: auth, docs upload, booking accept, live tracking, earnings - [] Admin Panel: guard verification, booking management, basic dashboard

Phase 2 — Safety & Trust (Months 5–6)

Goal: SOS system, incident reporting, reviews, call masking

Deliverables: - [] SOS panic button (button + shake trigger) - [] Emergency contact notifications (SMS + push) - [] Admin SOS control room (real-time WebSocket) - [] Check-in system for long sessions - [] Dead Man's Switch for guard offline detection - [] Incident reporting (user and guard side) - [] Post-session ratings and reviews - [] Call masking integration (Twilio Proxy) - [] Audio/video recording option

Phase 3 — Admin & Analytics (Months 7–8)

Goal: Full admin panel, payout automation, analytics

Deliverables: - [] Complete admin dashboard with live map - [] Full user and guard management with all actions - [] Automated guard payout system (Razorpay Payouts API) - [] Analytics module (booking trends, guard performance, revenue) - [] Broadcast notifications from admin panel - [] Financial reports and CSV/PDF export - [] Subscription/recurring booking support

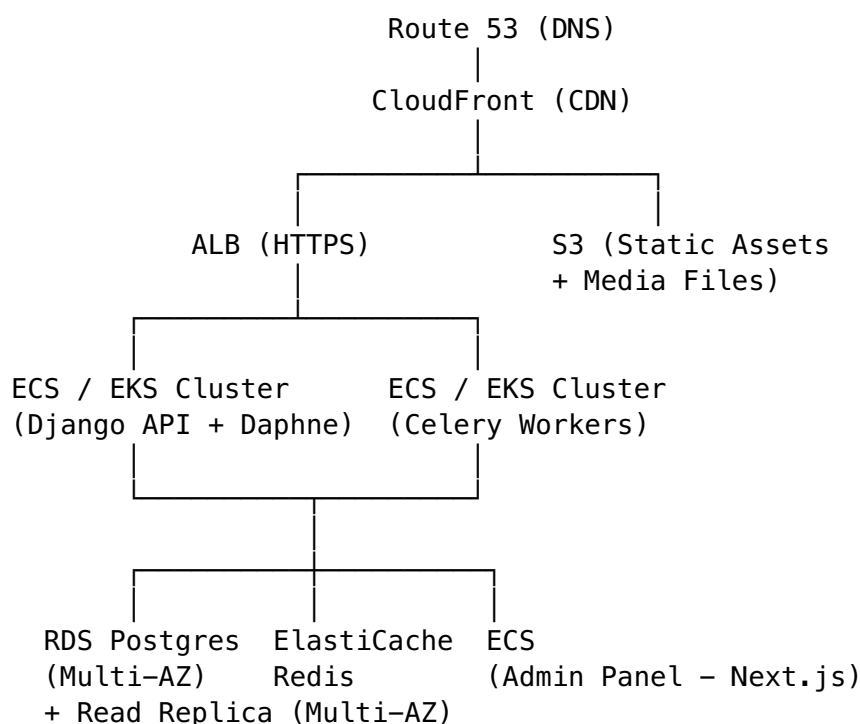
Phase 4 — Scale & Intelligence (Months 9+)

Goal: ML features, advanced pricing, expansion readiness

Deliverables: - [] ML-based guard matching (beyond proximity — behavior-based) - [] Dynamic surge pricing engine - [] Multi-city / multi-state expansion infrastructure - [] Elasticsearch-powered guard search - [] Referral program - [] Corporate accounts (B2B bookings) - [] API for third-party integrations (hotels, hospitals, corporates) - [] Multilingual app support (Hindi, regional languages)

14. Deployment Architecture

14.1 Infrastructure (AWS)



14.2 Docker Compose (Development)

```
services:
  db:
    image: postgis/postgis:16-3.4
    environment:
      POSTGRES_DB: bsecure
      POSTGRES_USER: bsecure
      POSTGRES_PASSWORD: dev_password

  redis:
    image: redis:7-alpine

  api:
    build: ./backend
    command: daphne -b 0.0.0.0 -p 8000 config.asgi:application
    depends_on: [db, redis]
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgis://bsecure:dev_password@db:5432/
        bsecure
      - REDIS_URL=redis://redis:6379/0

  celery_worker:
    build: ./backend
    command: celery -A celery_app worker -l info -Q
      high_priority,default,low_priority
    depends_on: [db, redis]

  celery_beat:
    build: ./backend
    command: celery -A celery_app beat -l info --scheduler
      django_celery_beat.schedulers:DatabaseScheduler
    depends_on: [db, redis]

  admin_panel:
    build: ./admin-panel
    ports:
      - "3000:3000"
```

14.3 CI/CD Pipeline (GitHub Actions)

On Pull Request:

1. Lint (flake8, black, isort)
2. Run unit tests (pytest)
3. Run security scan (bandit, safety)
4. Build Docker image

On Merge to main:

1. All PR checks
2. Run integration tests

3. Build and push Docker image to ECR
 4. Deploy to staging environment
 5. Run smoke tests on staging
 6. Manual approval gate
 7. Deploy to production (rolling update)
 8. Monitor error rate (auto-rollback if Sentry errors spike)
-

This document represents the complete system requirements for b-secure v1.0. It should be treated as a living document and updated as the product evolves.

Document Owner: b-secure Engineering Team Last Updated: May 2026